

Software Atelier 4: Software Engineering Project (Spring 2025)

Instructor: Dr. Andrea Mocci

TAs: Stefano Campanella, De Vita Gianmarco, Jamshidiyantehrani Masoud,
Puccinelli Niccolò, Lamberto Ragnolini

Version: 1.0 (March 4, 2025)



Preamble

This document is not final, and it is not perfect. It is part of the project to go through it and find areas where there might be need of clarifications (ambiguities, underspecified aspects, etc.) or where there are inconsistencies. The instructor is the responsible person to clarify those aspects, if needed.

1 Domain Description

SEWorld (codename) is a platform that supports teaching of software engineering concepts. In general, these topics match the one of this course. For this batch, we consider only:

1. Requirements Engineering;
2. Software Specifications;
3. (Black Box) Testing.

1.1 General concepts

The core of the domain for the platform is composed of the following concepts.

Topic A sub-topic of software engineering, as listed above. A topic is in general composed of materials and exercises.

Materials A collection of resources related to a topic. These resources can be links to external pages, pdfs, or internal pages specified in Markdown.

Exercise An exercise related to a topic. An exercise has a title and a description, and set of questions.

Question A question is a component of an exercise that requires an answer from a student. A question has a description, a percentage weight, and some form of correct answer which depends on the type of question. Examples of questions for this batch are true/false questions, multi-choice questions (from a set of pre-defined answers), and short answer questions, which are considered correct if they include a specific set of words.

The platform has two kinds of users with different roles, the teacher/instructor and the student.

2 User Stories (70 points)

1. As a user, I want to login using my SWITCH credentials, so that I can access SEWorld	10 pts
Once the SEWorld website is loaded, users can login via SWITCH Edu-Id credentials. ¹ If they are not logged in, the page will show a "login" button that will redirect the user to the eduid login page. The user needs to be a USI user.	
2. As a user, after logging in, I can get the list of topics.	5 pts
A logged-in user can see the list of the available topics. If the user is a student, each topic has a completion stage, depending on how many exercises have been successfully completed (among the available ones).	
3. As a user, I have my SEWorld profile	5 pts
In a dedicated page, users can see their profile, where they can inspect information about their basic corresponding identity at USI (e.g., username, email address). Moreover, they see some basic statistics about the topics they have completed in SEWorld . Profiles are only internally public, <i>i.e.</i> , they are available to all logged in users, but invisible if a user is not logged in.	
4. As a student, I can access the topic materials	5 pts
As a student, I can access the materials for each topic on a dedicated page.	
5. As an instructor, I can access the topic materials and edit them.	5 pts
As an instructor, I can access the materials for each topic on a dedicated page. I can add new types of content (e.g. an external link), edit the currently available materials, or delete some of them.	
6. As a student, I can access the topic exercises.	5 pts
A student can see the list of exercises and decide to access one of them. Students can access exercises that have already been solved as well.	
7. As a student, I can see the feedback over a solved exercise.	5 pts
A student access an exercise that has been already attempted and obtain feedback.	
8. As a student, I can attempt an exercise.	5 pts
As a user, I can attempt a novel exercise that I've never attempted or attempt again an exercise that I've tried already.	

¹ the same ones that you use for iCorsi.

9. As a teacher, I can add a new exercise	5 pts
As a teacher, I can add a new exercise, editing the basic attributes of it. The exercise is by default on a draft status and it is not available to the students.	
10. As a teacher, I can edit an exercise in draft status.	5 pts
As a teacher, I can edit an exercise in draft status by adding, deleting, and editing new questions. The questions are numbered and ordered in a dedicated exercise view.	
11. As a teacher, I can preview an exercise.	5 pts
As a teacher, I can preview an exercise and see it on a view that is identical (or very similar) to the one of a student that attempts it. I can also simulate the end of the attempt and obtain a feedback.	
12. As a teacher, I can remove the draft status and make an exercise available for student attempts.	5 pts
As a teacher, after finishing editing an exercise, I can remove the draft status and make it available for students to perform attempts. After an exercise has been made available, the exercise cannot be edited anymore, but it can be potentially removed.	
13. As a teacher, I can see the statistics of an exercise attempts by the students.	5 pts
As a teacher, when an exercise has been published and there are attempts by the students, I can see the resulting statistics (e.g., the number of students who did the attempts, how many questions were answered correctly, etc.)	

3 Development Requirements

A good implementation of the system requires a good comprehension of the requirements, a good object-oriented analysis step (for the conceptual model of your system), and a good design. You can use any of the methodologies that we listed during the lectures, including:

- constructing a good domain model of the system, including a phase where you do OO analysis on this document to understand which classes should be there (*e.g.*, Topic, Exercise, Question, Student, etc.)
- the use of UML diagrams - *e.g.*, , to depict scenarios, for example for what the frontend (our component that will have the role of the view) should do when a new simulation is created. Obviously, after the frontend and UML lectures.

This will help you to communicate with your team members, but also to communicate with us in case of need. Some requirements are imprecise, are probably ambiguous, and there may be many different ways to interpret them. This does not mean that all interpretations are valid: go back to your stakeholders (*i.e.*, us) if you are not sure what something means or entails.

All documentation should be on GitLab, *i.e.*, mostly on issues and merge requests. Later, if you want, you can have a dedicated project for documentation.

Given the fact that we still did not do the frontend lab lecture, you should start working on the backend. We will set up a base project which includes the code of Lab 02 and a basic CI pipeline. You should ensure that all your team members have seen the first two labs, and that they understand the concept of the lectures that involve coding.

During the first days of the project, you should mainly go over the document and analyze it. As a first step, you should open a Milestone for the first batch.

Then, other possible initial tasks include understanding in which order you can implement the user stories, if there is any dependency. Start opening issues that track the completion of each user story. Write in the issue description the user story itself and any possible other element of the document that involves them. You can assign a pair of people to one/two user stories. Their responsibility would be to derive dependencies and aspects of Section 1 that are related to the user stories. Then, as a team, you can go through them again and starting doing some OO analysis for the model (as in DDD), and start sketching the code for it (some user stories, though, do not involve the model).

Do not try to work on a entire user story in one single issue with its own merge request. It's better to create sub-issues for specific tasks related to one or more user stories: For example, modeling an exercise or a question would likely be needed for more user stories. Such smaller issues can have smaller merge requests that are easier to integrate.

When working on the domain model, you should be using what you know from the lectures on specifications, design, OOA, and DDD, and from the previous courses. A pair of team members can start investigating the logging in user story - this is nontrivial, but making it work is quite fun. Please make sure everybody understands what are the implications of this user story when completed - though its completion will require work on the frontend as well.

Do not try to finish the model before working on services and controllers. Try to start from a basic skeleton of classes in the domain model, even with minimal logic, and then try to understand which services, repositories, and controllers you will need. Focus testing on the domain model only - do not try to test services, controllers, or any springboot component.

Concerning some data types - try to **not reinvent the wheel** (in particular for dates and times).

4 Technical Details on Architectural Requirements

Logging in with Switch Edu-ID

For this task, it is extremely important that you do not follow random tutorials online, but that you follow these instructions. If you get stuck, you should ask for help from the instructor. It is important that the people assigned to this issue are skilled, and before integrating the login changes you explain to the other team members what are the implications - after these changes, it will be very difficult to test the requests to the backend with postman.

Logging in with Switch Edu-Id is supported through a series of protocols, mainly OAuth 2 and OpenID. These protocols are very complicated, and you are not supposed to manipulate them explicitly or to know many details about it. Fortunately, the Spring Security library that we added as a dependency in the second lab already implements the functionality to support logging in through these protocols.

First of all, you need to add the following dependencies to your project:

```
implementation 'org.springframework.boot:spring-boot-starter-oauth2-client'
implementation 'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
```

Then, add the following configuration to your application.properties file that is present in the main folder:

```
spring.security.oauth2.client.registration.switch-edu-id.client-id=test-client
spring.security.oauth2.client.registration.switch-edu-id.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.switch-edu-id.scope=openid
spring.security.oauth2.client.provider.switch-edu-id.issuer-uri=https://auth.dev.si.usi.ch/auth/realms/simurbs
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://auth.dev.si.usi.ch/auth/realms/simurbs
```

```
logging.level.org.springframework.security=DEBUG
```

The last one is particularly useful for debugging.

Trying the Log in process

In the first part of this batch, you are not supposed to construct a frontend for the application, but this would make the testing of the login with Gitlab a little hard to do with just postman.

First of all, try to require authentication for a specific route in the controller. You should modify the SecurityConfiguration class in your project. In the actual status, it permits all calls. You should change it by requiring - for some routes - to be authenticated with OAuth2. It is simple and it requires only to modify the SecurityConfiguration class, and in particular the filterChain method. If you want, you may have a look to some tutorials online, but please try not to get lost. If you're writing too much code for this, you are probably making some mistake. We can help.

When things work well, when you try to get from the browser a route that requires authentication, you should be redirected to the Switch Edu-Id page to log in. After a successful log in, you will see the right output for the route.

Retrieve user data after authentication

The first route that you could authenticate is for example one that will return some user data as it is coming from Switch Edu-Id.

It will look like this:

```
record SwitchEduIdUser(String fullName) {}

@GetMapping(path = "/me")
public ResponseEntity<SwitchEduIdUser> index(OAuth2AuthenticationToken authentication) {
    String fullName = authentication.getPrincipal().getAttribute("name");
    return ResponseEntity.ok(new SwitchEduIdUser(fullName));
}
```

You can imagine the authentication variable to represent the user data that you get after a successful login. This controller method will return the full name of the user. If you have properly authenticated this route (under the UserController), you will see your username.

Final Remarks

The user story 1 implies that your project will not manage authentication, and passwords, directly. This does not mean that you shouldn't have users in your domain model though. You need a way to 'map' a user from Switch Edu-Id (as you get from the OAuth2AuthenticationToken) to a user in the application you are developing. Switch Edu-Id users already have an identifier that you can essentially reuse - but again, this is something that you need to figure out.